Studio Nx

JS 124 — splitClip()

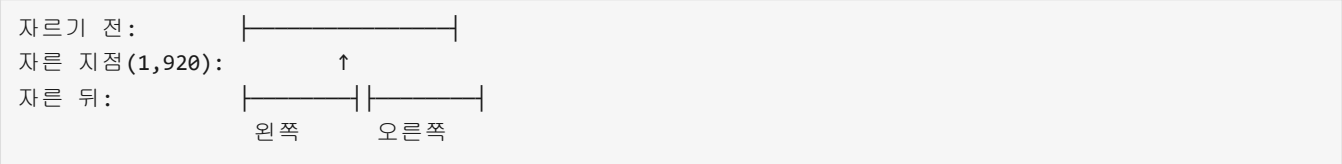
2026-10-14

JS 124 — `splitClip()` (클립 분할)

항목	내용
파일	`studionx/frontend/static/js/pages/sequencer-arrange.js`
작업	S1-06 타임라인: 열 번째 함수 (FR-208 분할)
작성	Claude (AI), 2026-10-14, 사용자 검토
브랜치	`feature/phase1/s1-06-timeline`
다음	클립 복제(Ctrl+D)는 `start + length` 한 줄이라 바로 이어서 넣습니다

1. 하는 일

클립을 **클릭 지점에서 둘로 가릅니다**. 자른 지점은 격자에 맞추고, 양쪽이 각각 **한 칸 이상**일 때만 가릅니다. 둘 중 하나라도 한 칸보다 짧으면 **가르지 않고 `null`**을 돌려줍니다.



자를 위치	한 칸	결과
1,920	960	왼쪽 {960, 960, 960} · 오른쪽 {1920, 960, 1920}
2,000	960	1,920로 붙어 위와 같음
960 (맨 앞)	960	`null` (왼쪽 0)
5,000 (끝 넘음)	960	`null` (오른쪽 0)
1,500	1 (Alt)	왼쪽 {960, 540, 960} · 오른쪽 {1500, 1380, 1500}
2,879	1 (Alt)	오른쪽이 1틱이라 분할됨

오디오는 자른 만큼 **오른쪽 토막의 오프셋이 커집니다**(원본에서 더 건너뜀). 오프셋 규칙(`sourceStartTick`)은 `trimClipLeft()`와 같습니다.

2. 코드

```
export function splitClip(startTick, lengthTick, atTick, stepTicks, sourceStartTick) {
  const at = snapTick(atTick, stepTicks);
  const leftLength = at - startTick;
  const rightLength = startTick + lengthTick - at;
  if (leftLength < stepTicks || rightLength < stepTicks) return null;
  const offset = (t) => (sourceStartTick == null ? 0 : t - sourceStartTick);
  return {
    left: { start: startTick, length: leftLength, offset: offset(startTick) },
    right: { start: at, length: rightLength, offset: offset(at) },
  };
}
```

최신 문법으로 쓰면 (공부용)

```
export const splitClip = (startTick, lengthTick, atTick, stepTicks, sourceStartTick) => {
  const at = snapTick(atTick, stepTicks);
  const leftLength = at - startTick;
  const rightLength = startTick + lengthTick - at;
  if (leftLength < stepTicks || rightLength < stepTicks) return null;
  const offset = (t) => (sourceStartTick == null ? 0 : t - sourceStartTick);
  return { left: { start: startTick, length: leftLength, offset: offset(startTick) }, right: { start: a
t, length: rightLength, offset: offset(at) } };
};
```

최신 문법	실제 코드	뜻
화살표 함수	`function` · 화살표	같은 동작입니다
`offset`을 화살표로 정의	그대로 사용	왼쪽·오른쪽 두 번 쓰려고 작은 함수로 묶었습니다

3. 왜 이렇게

선택	이유
자를 지점을 `snapTick()`으로 붙임	자른 자리가 격자 위에 있어야 두 토막이 다시 격자에 맞습니다
한 칸 미만이면 `null`	점처럼 작은 토막이 생기면 다시 잡기 어렵습니다. 못 가르면 아무 일도 안 일어난 게 낫습니다
`null`로 알림(오류 아님)	사용자가 클립 끝을 살짝 눌러 자르면 그냥 안 되는 게 자연스럽습니다. 빨간 오류가 뜨면 놀랍니다
오른쪽 오프셋 = `at - 원본시작`	왼쪽을 그대로 두고 오른쪽만 원본에서 더 건너뛰면 오디오가 제자리에 이어집니다
`sourceStartTick` 규칙 재사용	`trimClipLeft()`와 같은 규칙이라 MIDI(`null`)에서도 오프셋이 0으로 맞습니다
입력에 `offsetTick`이 없음	오프셋은 `start - 원본시작`으로 저절로 정해져 넣을 필요가 없습니다

4. 시험 (Node에서 직접 실행, 13개 통과)

확인	결과
가운데 자르기 · 격자로 붙어 자르기	✓
왼쪽·오른쪽 한 칸 미만 → `null` · 클립 밖 → `null`	✓
오디오 중간 시작(offset 480 → 오른쪽 1,440)	✓
MIDI(`null`) → 양쪽 offset 0	✓
Alt(step 1): 1,500·2,879 자르기, 끝(2,880) → `null`	✓
step 0·at `NaN` → 오류	✓